

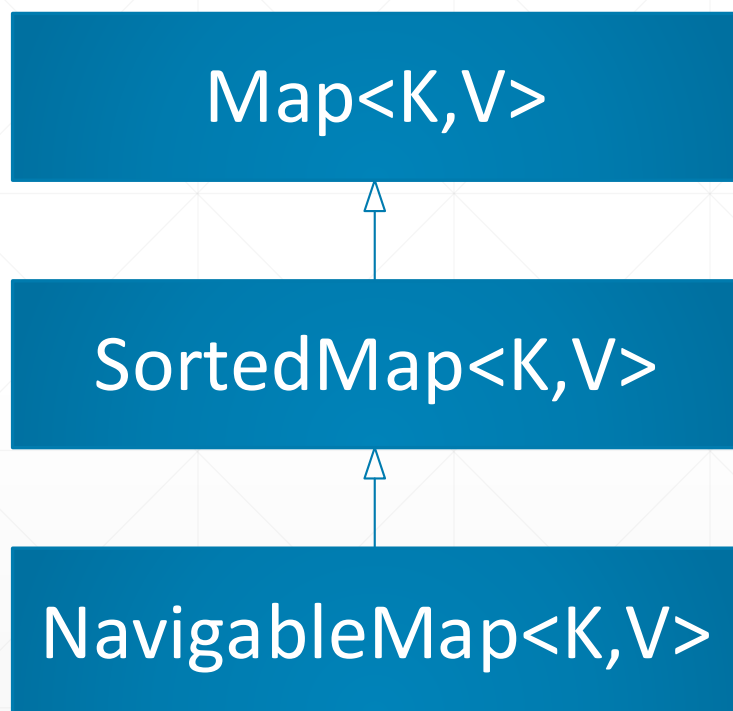
Map之二



有序的Map

北京理工大学计算机学院
金旭亮

JDK中的有序Map接口



```
▼ NavigableMap
  > SortedMap
    (m) lowerEntry(K): Entry<K, V>
    (m) lowerKey(K): K
    (m) floorEntry(K): Entry<K, V>
    (m) floorKey(K): K
    (m) ceilingEntry(K): Entry<K, V>
    (m) ceilingKey(K): K
    (m) higherEntry(K): Entry<K, V>
    (m) higherKey(K): K
    (m) firstEntry(): Entry<K, V>
    (m) lastEntry(): Entry<K, V>
    (m) pollFirstEntry(): Entry<K, V>
    (m) pollLastEntry(): Entry<K, V>
    (m) descendingMap(): NavigableMap<K, V>
    (m) navigableKeySet(): NavigableSet<K>
    (m) descendingKeySet(): NavigableSet<K>
    (m) subMap(K, boolean, K, boolean): NavigableMap<K, V>
    (m) headMap(K, boolean): NavigableMap<K, V>
    (m) tailMap(K, boolean): NavigableMap<K, V>
```

SortedMap的主要特点

- ➔ SortedMap中的元素按照Key进行排序，因此，可以找到第一个Key，也可以找到最后一个Key。

```
K firstKey();  
K lastKey();
```

- ➔ 给定一个或两个Key，能从Map中“切出”一块出来：

```
SortedMap<K, V> tailMap(E fromKey);  
SortedMap<K, V> headMap(E toKey);  
SortedMap<K, V> subMap(K fromKey, K toKey);
```

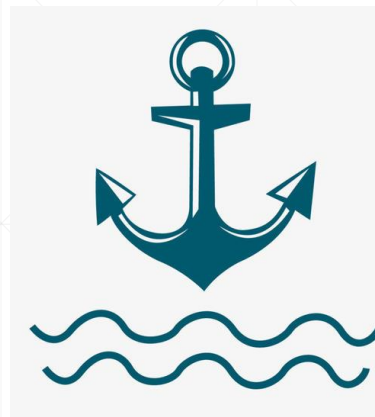
Map中的元素“结构”

我们已经知道，由于Map中保存的数据是“Key-Value”这样的数值对，所以，使用Map.Entry这样的类型来封装它们。

```
Map.Entry<K,V> firstEntry();  
Map.Entry<K,V> lastEntry();  
  
Map.Entry<K,V> pollFirstEntry();  
Map.Entry<K,V> pollLastEntry();
```

对于有序的Map，可以使用相应的方法访问或提取“第一个/最后一个”数值对，而poll*系列方法，会将相应数值对从集合中移除。

在Map中定位



```
Map.Entry<K,V> lowerEntry(K key);  
Map.Entry<K,V> higherEntry(K key);  
  
K lowerKey(K key);  
K higherKey(K key);
```

对于有序的Map，可以依据一个Key，找出它的上一个（比它小）的Key或Entry，也可以找出它的下一个（比它大）的Key或Entry。

“<=” 和 “>=”

```
Map.Entry<K,V> floorEntry(K key);  
Map.Entry<K,V> ceilingEntry(K key);
```

```
K floorKey(K key);  
K ceilingKey(K key);
```



floor: 地板在下，是“低”的，所以`floorKey(key)` 表示获取 “<=” 这个key的那个紧挨着的Key。



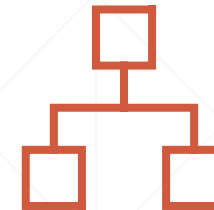
ceil: 天花板在上，是“高”的，所以，`ceilingKey(key)` 表示获取 “>=” 这个Key的那个紧挨着的Key。

反转顺序



```
NavigableMap<K, V> descendingMap()  
NavigableSet<K> descendingKeySet()
```

TreeMap



实现NavigableMap接口最常用的类型是TreeMap，它的使用方法与标准的Map没什么两样，比如也是使用put方法追加元素，使用get方法读取元素，使用remove方法移除元素。



TreeMap的独特之处在于：它内部基于红黑树（red-black tree）对Key进行排序。因此，它的元素是“有序”的。

UseTreeMap.java

```
class R implements Comparable {
    int value;

    public R(int value) {
        this.value = value;
    }

    public String toString() {
        return "R(value属性:" + value + ")";
    }

    //重写了equals方法, 如果value属性相等返回true
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj != null && obj.getClass() == R.class) {
            R r = (R) obj;
            if (r.value == this.value) {
                return true;
            }
        }
        return false;
    }
}
```

为了展示TreeMap的特性, 专门设计了一个R类, 这个R类很特殊, 重写了equals和compareTo方法。

```
//重写了compareTo(Object obj)方法, 如果value属性相等返回0;
public int compareTo(Object obj) {
    R r = (R) obj;
    if (this.value > r.value) {
        return 1;
    } else if (this.value == r.value) {
        return 0;
    } else {
        return -1;
    }
}
```

使用TreeMap示例

示例输出：

TreeMap集合中的所有元素：

{R(value属性:-5)=Key值为-5的value,
R(value属性:3)=Key值为3的value,
R(value属性:9)=Key值为9的value}

第一个Entry对象：

R(value属性:-5)=Key值为-5的value

最后一个key值：

R(value属性:9)

比new R(2)大的最小key值：

R(value属性:3)

比new R(2)小的最大的key-value对：

R(value属性:-5)=Key值为-5的value

该TreeMap的子TreeMap：

{R(value属性:3)=Key值为3的value}

```
public class UseTreeMap {  
    public static void main(String[] args) {  
  
        var treeMap = new TreeMap<R, String>();  
        treeMap.put(new R( value: 3), "Key值为3的value");  
        treeMap.put(new R( value: -5), "Key值为-5的value");  
        treeMap.put(new R( value: 9), "Key值为9的value");  
        System.out.println("TreeMap集合中的所有元素: \n" + treeMap);  
  
        //返回该TreeMap的第一个Entry对象  
        System.out.println("\n第一个Entry对象: " + treeMap.firstEntry());  
        //返回该TreeMap的最后一个key值  
        System.out.println("最后一个key值:" + treeMap.lastKey());  
        //返回该TreeMap的比new R(2)大的最小key值。  
        System.out.println("比new R(2)大的最小key值:" +  
            treeMap.higherKey(new R( value: 2)));  
        //返回该TreeMap的比new R(2)小的最大的key - value对。  
        System.out.println("比new R(2)小的最大的key - value对:" +  
            treeMap.lowerEntry(new R( value: 2)));  
        //返回该TreeMap的子TreeMap  
        System.out.println("该TreeMap的子TreeMap:" +  
            treeMap.subMap(new R( value: -1), new R( value: 4)));  
    }  
}
```

小结



TreeMap是JDK中功能最为强大的Map，其主要特点是它所包容的key-value对总是处于有序状态，无须专门进行排序操作，活用这一特性，有时会带来方便。

当向TreeMap中插入元素时，其内部可能会进行频繁的“树”重构操作，因此，其性能不如标准的HashMap。